

Report Homework Assignment 1

Mursal Stella Mohammadi

September 17, 2026

1 Introduction

The goal is to build a password recovery tool for an AES-256 encrypted file. The work has three parts. A sequential brute-force version, an MPI-parallelized version, and one optimization of the parallel version.

There is the encrypted file made with AES-256, a fixed salt for PBKDF2, and an Initialization Vector. The password is known to come from a 62-character set: digits (0–9), lowercase letters (a–z), and uppercase letters (A–Z). Along with the ciphertext, the SHA-512 checksum of the original plaintext is given. This checksum is used to check if a decryption worked.

To create more test cases, the provided *encrypt* executable is used, which produces both the *.enc* and *.sha512* files.

2 Sequential solution

charset.h and charset.c

I moved `CHARSET_SIZE` and `CHARSET` into `charset.h/charset.c` to avoid duplication between `main.c` and `brute_seq.c`.

Listing 1: `charset.h`

```
#define CHARSET_SIZE 62
extern char CHARSET[CHARSET_SIZE];
```

Makefile

`charset.h` is included in `main.c` and `brute_seq.c`, so the Makefile was adjusted:

Listing 2: Makefile

```
...
main: main.c aes256.c utils.c charset.c
    gcc $^ -o $@ -lcrypto
```

```
brute: brute_seq.c aes256.c utils.c charset.c
      gcc $^ -o $@ -lcrypto
...
```

brute_seq.c

The program takes three arguments:

- `password_length` (length to brute force)
- `.enc` file path
- `.sha512` file path

I first tried searching all lengths up to a maximum, but this grows by a factor of 62 per length, so I switched to a fixed length per run.

```
if (argc != 4)
{
    printf("Missing something!\n%s<password_length>
--<.enc_file_path>-<.sha512_file_path>\n", argv[0]);
    return 1;
}
```

Buffers are allocated dynamically:

```
uint8_t key[AES_256_KEY_LENGTH];
uint8_t *ciphertext = malloc(MAX_FILE_SIZE_B);
uint8_t *plaintext = malloc(MAX_FILE_SIZE_B);
uint8_t plaintext_checksum[SHA512_DIGEST_LENGTH];
uint8_t computed_checksum[SHA512_DIGEST_LENGTH];
```

After loading both files (with checks), I verified the pipeline with the password "abc" on the `myfile` test files.

total (number of candidates, 62^{length}) is computed with a loop using `int64_t` to avoid overflow. Timing starts before the main loop:

```
clock_gettime(CLOCK_MONOTONIC, &start);
```

Password finding

The counter is treated as an index in base-62. Each digit maps to a `CHARSET` character. Example: index 512 $\rightarrow$ "8g".

- $512 / 62 = 8$ rest 16 $\rightarrow$ `CHARSET[16] = 'g'`
- $8 / 62 = 0$ rest 8 $\rightarrow$ `CHARSET[8] = '8'`

So the password is decoded right to left.

```

for (int64_t counter = 0; counter < total; counter++)
{
    int64_t n = counter;
    for (int pos = password_length - 1; pos >= 0; pos--)
    {
        password[pos] = CHARSET[n % CHARSET_SIZE];
        n /= CHARSET_SIZE;
    }
    password[password_length] = '\0';
}

```

Then the candidate runs through the pipeline (from `main.c`): `pbkdf2` → `decrypt` → `sha512sum` → `sha512cmp`.

```

pbkdf2(password, password_length, key);
int32_t plaintext_length = decrypt(ciphertext,
ciphertext_length, key, plaintext);
if (plaintext_length >= 0)
{
    sha512sum(plaintext, plaintext_length, computed_checksum);
    if (!sha512cmp(plaintext_checksum, computed_checksum))
    {
        plaintext[plaintext_length] = '\0';
        printf("Encrypted file contains: %s\n", plaintext);
        printf("The password to the file is: %s\n", password);
    }
}

```

On success: stop timer, free buffers, return 0. On failure: print message, stop timer, free buffers, return 1.

Executing brute

Example (a 4-character test file is stored in `test1`):

```

make brute
./brute 3 ./files/myfile.enc ./files/myfile.sha512

```

3 Parallel solutions

Before running `mainMPI` or `optMPI`, I update the hostfile with the currently available nodes on the SSH server:

```

‘/share/ifi/available-nodes.sh’

```

3.1 mainMPI

`mainMPI.c` is my MPI-parallelized version of the brute-force cracker. It takes three arguments:

```

mainMPI <password_length> <path_to_.enc_file> <path_to_.sha512_file>

```

Like the sequential version, it tries every password of the given length over the 62-character `CHARSET`, derives an AES key via PBKDF2, decrypts the ciphertext, and compares the SHA-512 of the decrypted plaintext against the known checksum. On a match, it prints the plaintext and password and stops.

Parallelization strategy

The search space has size $total = 62^L$. I split it into blocks by process count:

```
chunk = total / size
start = chunk * r
end   = (r == size-1) ? total : start + chunk
```

The last rank takes the remainder, so all candidates are covered exactly once. Every rank reads the same files and runs the brute-force loop on its own rank.

Once a rank finds the password, all others must stop. The finding rank sets `found = 1`, sends the flag with `MPI_Isend` to every other rank, and calls `MPI_Waitall` before breaking. Every other rank polls with `MPI_Iprobe` every 100 iterations. I placed an `MPI_Barrier` before the timer and one after the loop, so timing is fair. And only rank 0 prints the time.

Running:

```
mpirun -np <P> ./mainMPI <password_length> <enc_file> <sha_file>
```

Example, 8 processes, 4-character password:

```
mpirun -np 8 --hostfile hostfile ./mainMPI 4
./files/test4/test4.enc ./files/test4/test4.sha512
```

Expected output:

```
Looking for password with length = 4 with 8 processes
Encrypted file contains: Somefile wtf
```

```
Rank 0: The password to the file is: 6gsF
```

```
Rank 6: Password not found with given length 4 or in this rank.
```

```
Rank: 6 Time: 10.862 seconds
```

```
Rank: 4 Time: 10.857 seconds
```

```
...
```

Every rank that does not find the password prints:

```
Password not found with given length ... or in this rank.
```

3.2 Optimization of mainMPI

My optimized version `optMPI.c` targets three bottlenecks in `mainMPI.c`:

- Independent file I/O (all ranks) $\rightarrow$ Rank 0 I/O + `MPI_Bcast`
- Division-based string lookup $\rightarrow$ Odometer advancement
- Blocking `MPI_Waitall` $\rightarrow$ Non-blocking `MPI_Isend`

1. Broadcast Optimization

In `mainMPI`, every rank reads the `.enc` and `.sha512` files itself, causing filesystem rivalry. In `optMPI`, I centralize I/O on rank 0 and distribute data via `MPI_Bcast`.

2. Odometer

In `mainMPI`, the counter is converted to a string each iteration via modulo and division. In `optMPI`, I compute the starting string once, then advance odometer-style:

```
for (int pos = password_length - 1; pos >= 0; pos--) {
    password_index[pos]++;
    if (password_index[pos] < CHARSET_SIZE) {
        password[pos] = CHARSET[password_index[pos]];
        break;
    }
    password_index[pos] = 0;
    password[pos] = CHARSET[0];
}
```

3. Non-blocking MPI Isend

In `mainMPI`, the finding rank waits on `MPI_Waitall`. In `optMPI`, I replace it with non-blocking handle cleanup (`MPI_Request_free(&req)`). Ranks poll with `MPI_Iprobe` every 100 iterations to reduce sync overhead.

Example run:

```
mpirun -np 8 ./optMPI 5 ./files/test3/test3.enc ./files/test3/test3.sha512
```

3.3 Optimization Analysis: mainMPI vs. optMPI

I benchmarked both versions under exhaustive search ($L = 4$) with the same process counts.

At low process counts, both implementations show near-identical execution times.

MPI Ranks (p)	mainMPI Time (s)	optMPI Time (s)	Performance Gain
1	172.04	171.31	+0.42%
4	42.77	42.89	-0.28% (Noise)
8	21.48	21.47	+0.05%
12	14.41	14.33	+0.56%
16	10.78	10.81	-0.28% (Noise)
20	10.19	8.62	+15.38%
24	9.65	7.20	+25.39%

Table 1: Comparative performance between `mainMPI` and `optMPI` ($L = 4$).

4 Performance evaluation

I ran all benchmarks on the shared university SSH cluster. Before running, I checked the hardware with `lscpu` and the system load with `uptime`.

Hardware and System Environment

- **CPU:** Intel Xeon Silver 4214R @ 2.40GHz
- **Topology:** 2 sockets, 24 physical cores (12 per socket, SMT off)
- **Memory:** 2 NUMA nodes, shared memory (OpenMPI / UCX)
- **Load at test:** load average: 0.02

Performance Metrics

I use three metrics:

Metric	Formula	Description
Execution Time (T_p)	<code>clock_gettime</code>	Elapsed time in seconds (rank 0).
Speedup (S_p)	$S_p = \frac{T_1}{T_p}$	Single-process time divided by p -process time.
Efficiency (E_p)	$E_p = \frac{S_p}{p}$	How close to ideal linear speedup ($0 \leq E_p \leq 1$).

Table 2: Definitions of the parallel scaling metrics [1].

Ideally, speedup is linear ($S_p = p$) and efficiency stays at 1.0. In practice it drops because of serial setup (file I/O, SHA setup) and polling overhead (`MPI_Iprobe`).

Handling Search Space Variance

Where the password sits in the search space strongly affects runtime. To make a fair test, I use the following strategy

- I pass length 5 while the real password has length 4. No match is possible, so every rank searches its full chunk ($62^5 = 916,132,832$ candidates). This removes positional bias.

Strong Scaling Analysis

Fixed search space $L = 4$, $C = 62$, varying $p \in \{1, 4, 8, 12, 16, 24\}$.

Ranks (p)	Total Space (N)	Runtime T_p (s)	Speedup (S_p)	Efficiency (E_p)
Seq Baseline	14,776,336	142.38	$1.00\times$	100.00%
1	14,776,336	171.31	$0.83\times$	83.11%
4	14,776,336	42.89	$3.32\times$	82.99%
8	14,776,336	21.47	$6.63\times$	82.89%
12	14,776,336	14.33	$9.94\times$	82.80%
16	14,776,336	10.81	$13.17\times$	82.32%
20	14,776,336	8.62	$16.52\times$	82.59%
24	14,776,336	7.20	$19.78\times$	82.39%

Table 3: Strong scaling results against the sequential baseline ($T_{\text{seq}} = 142.38\text{s}$).

Analysis

- Strong scaling shows how runtime drops when I split a fixed workload over more ranks.
- Against the sequential baseline, I see a fixed $\sim 17\%$ MPI overhead at $p = 1$. Beyond that, scaling is close to linear: efficiency stays near 82.4% and I reach $19.78\times$ at 24 ranks.

```
[mumoh5118@ificluster code]$ ./brute 4 ./files/myfile.enc ./files/myfile.sha512
Looking for password with length: 4
Password not found with given length: 4
Time: 142.380 seconds
```

Figure 1: Output `./brute`

Weak Scaling Analysis

For weak scaling I wrote `weakMPI.c`. I could not scale by increasing L , because each step in L multiplies the space by 62 instead of growing linearly.

Results in Table 4 show very stable scaling.

```

• [mumoh5118@ifcluster code]$ mpirun -np 1 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 1 processes
  Time: 172.035339441 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 4 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 4 processes
  Time: 42.766873692 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 8 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 8 processes
  Time: 21.478912838 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 12 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 12 processes
  Time: 14.409804943 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 16 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 16 processes
  Time: 10.784418511 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 20 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 20 processes
  Time: 10.190485344 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 24 --hostfile hostfile ./mainMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 24 processes
  Time: 9.654111913 seconds

```

Figure 2: Output ./mainMPI

```

• [mumoh5118@ifcluster code]$ mpirun -np 1 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 1 processes
  Time: 171.314238229 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 4 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 4 processes
  Time: 42.885478407 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 8 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 8 processes
  Time: 21.470184688 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 12 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 12 processes
  Time: 14.327889434 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 16 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 16 processes
  Time: 10.810391855 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 20 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 20 processes
  Time: 8.623037900 seconds
• [mumoh5118@ifcluster code]$ mpirun -np 24 --hostfile hostfile ./optMPI 4 ./files/myfile.enc ./files/myfile.sha512
  Looking for password with length = 4 with 24 processes
  Time: 7.200131468 seconds

```

Figure 3: Output ./optMPI

Ranks (p)	Total Space (N)	Work / Rank	Runtime T_p (s)	Weak Efficiency (E_w)
1	5,000,000	5,000,000	58.19	100.00%
4	20,000,000	5,000,000	58.04	100.25%
8	40,000,000	5,000,000	58.18	100.01%
12	60,000,000	5,000,000	58.84	98.89%
16	80,000,000	5,000,000	58.17	100.03%
20	100,000,000	5,000,000	58.23	99.92%
24	120,000,000	5,000,000	58.34	99.74%

Table 4: Weak scaling with a fixed workload of 5,000,000 candidates per rank.

Analysis

- From $p = 1$ to $p = 24$, runtime stays in a narrow range of 58.04s to 58.84s.
- Weak efficiency stays above 98.8% everywhere, and even at 24 ranks $E_w(24) = 99.74\%$.

```

• [mumoh5118@ificluster code]$ mpirun -np 1 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 5000000 with 1 ranks
Weak Scaling Execution Time: 58.185710123 seconds
• [mumoh5118@ificluster code]$ mpirun -np 4 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 20000000 with 4 ranks
Weak Scaling Execution Time: 58.038670830 seconds
• [mumoh5118@ificluster code]$ mpirun -np 8 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 40000000 with 8 ranks
Weak Scaling Execution Time: 58.181694303 seconds
• [mumoh5118@ificluster code]$ mpirun -np 12 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 60000000 with 12 ranks
Weak Scaling Execution Time: 58.840473883 seconds
• [mumoh5118@ificluster code]$ mpirun -np 16 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 80000000 with 16 ranks
Weak Scaling Execution Time: 58.167285848 seconds
• [mumoh5118@ificluster code]$ mpirun -np 20 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 100000000 with 20 ranks
Weak Scaling Execution Time: 58.231285193 seconds
• [mumoh5118@ificluster code]$ mpirun -np 24 --hostfile hostfile ./weakMPI 5 ./files/test4/test4.enc ./files/test4/test4.sha512
WEAK SCALING RUN: Length = 5, Work per rank = 5000000, Total space = 120000000 with 24 ranks
Weak Scaling Execution Time: 58.337767175 seconds

```

Figure 4: Output of weak scaling

5 Discussion

In theory, speedup is limited by the part of the code that runs sequentially. In my `optMPI`, the compute part is parallel, but there is still some fixed overhead like the periodic `MPI_Iprobe` calls. This could explain why running with 1 rank (171.31s) is slower than the pure sequential baseline (142.38s), giving only $E_1 = 83.11\%$.

But past that starting offset, scaling is almost linear up to 24 ranks ($S_{24} = 19.78\times$). And in weak scaling, where I keep the workload per rank fixed (5×10^6 candidates), efficiency stays almost perfect ($E_w \geq 98.89\%$). So communication overhead stays flat, even when the total space grows to 1.2×10^8 .

Design Choices

- Search space grows fast (62^L). Each extra password character multiplies the work by 62. So testing anything above $L = 5$ takes too long. That is why I chose $L = 4$ for the scaling tests.
- Polling frequency (`reset_countdown = 100`). Checking for a stop signal with `MPI_Iprobe` is a trade-off. If I check too often, I steal CPU time from real work. If I check too rarely, other ranks keep working after the password is already found. Checking every 100 iterations is a good middle ground.
- In `mainMPI`, every rank reads the input files by itself. With many ranks reading at once, the filesystem becomes the bottleneck. In `optMPI`, I let only Rank 0 read the files and then share them with `MPI_Bcast`.

5.1 Future Improvements

Even though `optMPI` is efficient, using MPI between nodes and OpenMP threads inside each node could make it faster.

6 Conclusion

I parallelized and optimized an AES-256 password cracker with MPI. The parallel versions scale well. By removing the I/O bottleneck, switching to an odometer style candidate loop, and fine-tuning the non-blocking early-exit check, `optMPI` reaches a speedup of $19.78\times$ and weakscale efficiency of $E_w \geq 98.89\%$ at 24 ranks. Cracking passwords of length $L \geq 5$ will need acceleration.

References

- [1] Harmanani, H. M.: CSC 447: Parallel Programming for Multi-Core and Cluster Systems — Lecture 04: Performance Analysis. Lecture notes, Lebanese American University, Spring 2021. <https://harmanani.github.io/classes/csc447/Notes/Lecture04.pdf>